Project Summary: Indian Stock Market Prediction using Random Forest Regressor

This notebook demonstrates an end-to-end process for predicting stock 'Close' prices using historical Indian Stock Market data. The methodology involves data loading, comprehensive preprocessing, model training with a Random Forest Regressor, rigorous evaluation, and a practical demonstration of making predictions on new data.

1. Data Loading and Initial Inspection

The project began by loading the Indian Stock Market Data.csv dataset into a pandas DataFrame. Initial steps included:

- **Importing Libraries**: Essential libraries such as pandas, numpy, matplotlib.pyplot, seaborn, sklearn.model_selection, sklearn.preprocessing, sklearn.ensemble, and sklearn.metrics were imported.
- **Loading Data**: The dataset was loaded using pd.read_csv('/content/Indian Stock Market Data.csv').
- **Date Conversion**: The 'Date' column was converted to datetime objects using pd.to_datetime(df['Date']) to facilitate time-series analysis.
- **Initial Data View**: The df DataFrame was displayed to show its structure and content.

---

2. Data Preprocessing

Prior to model training, the data underwent several crucial preprocessing steps to ensure quality and suitability for the machine learning model:

- **Missing Value Check**: df.isnull().sum() was used to identify columns with missing values. It was observed that 'Open', 'High', 'Low', 'Close', 'Adj Close', and 'Volume' columns had one missing value each.
- **Missing Value Imputation**: Missing numerical values were imputed using the mean of their respective columns with df.fillna(df.mean(numeric_only=True), inplace=True). This strategy helps maintain data integrity without significantly altering the distribution.
- **Categorical Feature Encoding**: The 'Company' column, which is categorical, was converted into numerical representations using a manual mapping. Each company name was assigned a unique integer ID from 1 to 50. This is a necessary step for machine learning models that require numerical inputs.

- **Data Information and Shape**: df.info() and df.shape were used to verify the data types, non-null counts, and dimensions of the DataFrame after preprocessing, confirming all columns were of appropriate types and no missing values remained.
- **Descriptive Statistics**: df.describe() provided a statistical summary of the numerical columns, offering insights into their distribution, central tendency, and spread.

---

3. Feature and Target Separation

The dataset was prepared for model training by separating the features (independent variables) from the target variable (dependent variable):

- **Features (X)**: All columns except 'Close' (the target) and 'Date' (which is not directly used as a numerical feature in this model) were selected as features. X = df.drop(['Close', 'Date'], axis=1).
- **Target (y)**: The 'Close' price was designated as the target variable: y = df['Close'].

4. Data Splitting

The prepared dataset was split into training and testing sets to evaluate the model's performance on unseen data:

- **Train-Test Split**: The data was divided into 70% for training and 30% for testing using train_test_split(X, y, test_size=0.3, random_state=42). A random_state was set for reproducibility.

---

5. Model Training

A Random Forest Regressor was chosen for its robustness and ability to handle complex relationships within the data:

- **Model Initialization**: An instance of RandomForestRegressor was created with random_state=42 for consistent results: rf_model = RandomForestRegressor(random_state=42).
- **Model Fitting**: The model was trained on the training data using rf_model.fit(X_train, y_train).

6. Model Evaluation

After training, the model's performance was assessed on the test set:

- **Prediction on Test Set**: The trained model made predictions on the unseen test features: y_pred = rf_model.predict(X_test).
- **Accuracy Measurement**: The r2_score was used to evaluate the model's accuracy, yielding an impressive score of 0.9999717761578077, indicating a very strong fit to the data.
- **Score on Test and Train Sets**: The rf_model.score() method confirmed the R-squared values for both the test set (0.9999717761578077) and the training set (0.9999955510970683), suggesting excellent performance and minimal overfitting.

---

7. Prediction on New Data

Finally, the notebook demonstrates how to use the trained model to predict the 'Close' price for new, user-provided stock data:

- **User Input**: The model prompts the user to input values for 'Open', 'High', 'Low', 'Adj Close', 'Volume', and 'Company ID'.
- **Data Preparation for Prediction**: The input values are structured into a pandas DataFrame, ensuring its columns match the training data's feature set. This step is crucial for the model to correctly interpret the new data.
- **Making Prediction**: The rf_model.predict() method is called on the prepared new data, and the predicted 'Close' price is then displayed to the user, formatted to two decimal places. This showcases the practical application of the trained model in forecasting stock prices.
